INFORME TÉCNICO DE IMPLEMENTACIÓN Y ARQUITECTURA POO - ACTIVIDAD 8

Materia: Laboratorio de Programación (LPR) — 5° Año 3° División

Institución: E.E.S.T. N° 1 "Eduardo Ader" — Vicente López

Asignatura: Laboratorio de Programación (LPR)

Docente Evaluador: Prof. York Mansilla

Autora / Alumna: Sofía Mojica

Fecha: 24 de octubre de 2026 | **Versión:** v1.0.0

---

1. INTRODUCCIÓN

Este informe detalla el diseño, la estructura e implementación de la Actividad 8 del Laboratorio de Programación. La práctica marca el paso de la programación estructurada —basada en el agrupar variables de distinto tipo con struct— hacia la Programación Orientada a Objetos (POO) en C++. A través del desarrollo de un simulador de combate espacial en consola, se analiza cómo el uso de clases y el encapsulamiento de datos permiten crear programas más ordenados, seguros y fáciles de mantener, sentando las bases arquitectónicas para el Proyecto Integrador Anual (P.I.A.).

2. DESARROLLO

Definiciones Técnicas:

- **Clase (class):** Plantilla o modelo que reúne en un solo bloque los datos (atributos) y las funciones que operan sobre ellos (métodos), definiendo la estructura y el comportamiento de los objetos que se creen a partir de ella.
- **Encapsulamiento:** Principio de diseño que consiste en ocultar la estructura interna y las variables de un objeto para evitar modificaciones no autorizadas desde fuera, permitiendo interactuar con él solo mediante funciones públicas bien definidas.
- **Constructor:** Función especial de una clase que tiene su mismo nombre y se ejecuta automáticamente al crear un objeto. Su función es inicializar las variables internas con valores seguros desde el primer momento.

Resolución del Cuestionario Técnico de Control

Pregunta 1

¿Cuál es la principal diferencia técnica entre un struct (visto en la Actividad 7) y una clase, especialmente respecto a la sección private y public?

Respuesta:

La diferencia principal radica en los permisos de acceso que el compilador aplica por defecto:

- En una estructura (struct), todos sus miembros son **públicos (public)** por defecto, lo que permite que cualquier parte del programa lea o cambie sus variables directamente.
- En una clase (class), todos sus miembros son **privados (private)** por defecto, lo que prohíbe el acceso directo desde el exterior y obliga a declarar explícitamente qué partes serán públicas.

Pregunta 2

¿Por qué es peligroso dejar la variable de estado (vidas/salud) como pública? ¿Qué desastre podría causar un programador novato desde el main()?

Respuesta:

Dejar variables críticas como públicas rompe el encapsulamiento y quita el control sobre los datos. Desde la función main(), se podrían asignar valores ilógicos o negativos por error (por ejemplo: jugador.vidas = -500;). Esto saltearía las validaciones del juego, alterando la lógica de combate y pudiendo provocar fallos inesperados o el cierre repentino del programa.

Pregunta 3

Explicá con tus palabras para qué sirve el bloque Nave(int _x, int _y, int _vidas) dentro del código.

Respuesta:

Este bloque es el **constructor** de la clase Nave. Su función es asegurar que, al momento de crear la nave en el juego, sus coordenadas iniciales en pantalla (_x, _y) y su nivel de salud (_vidas) comiencen con valores válidos. Esto evita trabajar con datos basura en memoria y garantiza que la nave se dibuje en la posición correcta desde el inicio.

Pregunta 4

Explicá con tus palabras qué funcionalidad elegiste agregar de las 15 opciones y en qué partes del código tuviste que insertarla para que funcione.

Respuesta:

Elegí agregar un **Sistema de Puntuación Acumulativa (Score)** para registrar los puntos en tiempo real:

1. Se declaró la variable de tipo entero puntos (inicializada en 0) dentro de la función main().
2. Se modificó el bloque que detecta el choque entre los disparos y los asteroides.
3. Al confirmarse un impacto, el programa suma puntos (puntos += 100;) y actualiza el valor en la pantalla usando la función gotoxy(2, 2) para posicionar el texto en la parte superior del tablero.

3. CONCLUSIÓN

El desarrollo de este simulador confirmó que la Programación Orientada a Objetos y el encapsulamiento son fundamentales para desarrollar software confiable. Proteger las variables mediante acceso privado e interactuar con ellas mediante métodos públicos y constructores evita el uso indebido de datos globales, reduce errores en ejecución y proporciona una estructura limpia y escalable para encarar el Proyecto Integrador Anual.

---

4. REFERENCIAS

- cplusplus.com. (2026). *Clases y Objetos en C++ y directivas de control de flujo*. cplusplus.com
- GeeksforGeeks. (2026). *Modificadores de acceso públicos y privados en arquitecturas C++*. geeksforgeeks.org

5. REPOSITORIOS

- https:/github.com/Ailuna77/juegodenaves.git
- https://gitlab.com/Ailuna77/juegonaves.git